GENAI HANDS-ON • UNIT 2

Unit 2 Observation Report

This report contains my observations and understanding from the Unit 2 hands-on notebooks and assignment.

Name: Amitesh Sinha

SRN: PES2UG23CS054

Section: A

Course: GenAI Hands-on Unit 2

1. Notebook Coverage

This observation report is based on all the main notebook files and the assignment work from Unit 2. I included the ideas and experiments that were repeated across the hands-on exercises so the report reflects the full unit instead of only one notebook.

- 1_LangChain_Foundation.ipynb: setup, tokens, temperature, prompt templates, output parsers, and LCEL chaining.
- 2_Prompt_Engineering.ipynb: prompt structure, CO-STAR, zero-shot, few-shot, and advanced prompt templates.
- 3_Advanced_Prompting.ipynb: Chain of Thought, Tree of Thoughts, and Graph of Thoughts.
- 4_RAG_and_Vector_Stores.ipynb: embeddings, cosine similarity, FAISS, RAG pipeline, and indexing algorithms.
- Assignment_MOE_Solution.ipynb: Mixture of Experts routing, specialized prompts, orchestration, and tool-use extension.

2. LangChain Foundation

I understood LangChain as a framework layer that sits between my code and the model provider. The main practical advantage is that it standardizes model access, so changing providers later becomes easier. Instead of rewriting raw API logic, I can swap the model wrapper and keep the rest of the pipeline mostly unchanged.

What I learned

- Tokens are the basic units the model reads, not full words.
- Context window means how much the model can remember in one interaction.
- API keys should be loaded securely with dotenv or getpass, not hardcoded.

My takeaway

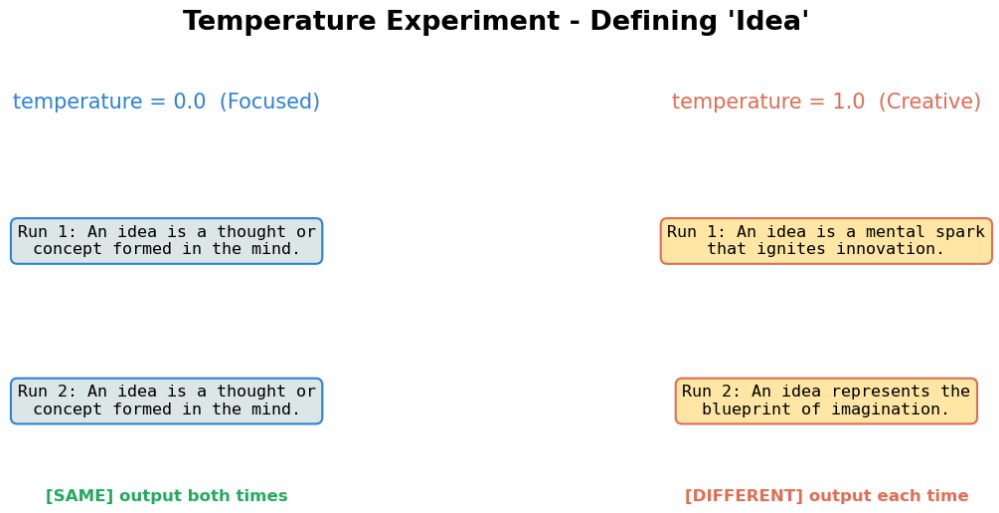
- LangChain reduces future migration effort.
- The setup pattern is reusable across notebooks.
- Security and clean environment setup matter even in experiments.

Notebook 1 - LangChain Setup & Model Init

Code:

```
from dotenv import load_dotenv
load_dotenv()

import getpass, os
from langchain_google_genai import ChatGoogleGenerativeAI

if "GOOGLE_API_KEY" not in os.environ:
    os.environ["GOOGLE_API_KEY"] = getpass.getpass("Enter your Google API Key: ")

llm_focused = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.0)
llm_creative = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=1.0)
```

Notebook screenshot: secure environment setup and model initialization pattern used across Unit 2.



Notebook screenshot: comparing focused and creative temperature settings helped me see consistency versus randomness in practice.

3. Prompts, Roles, and LCEL

One of the clearest lessons for me was that prompts are not just text. They can be structured by role, template, and parser. System messages give strong control over behavior, and prompt templates are safer than building strings manually. LCEL then connects the full pipeline in a clean way.

> *My understanding: LCEL feels like Unix piping for LLM workflows. Each block does one job, and the output flows to the next block.*

Notebook 1 - System Message Experiment

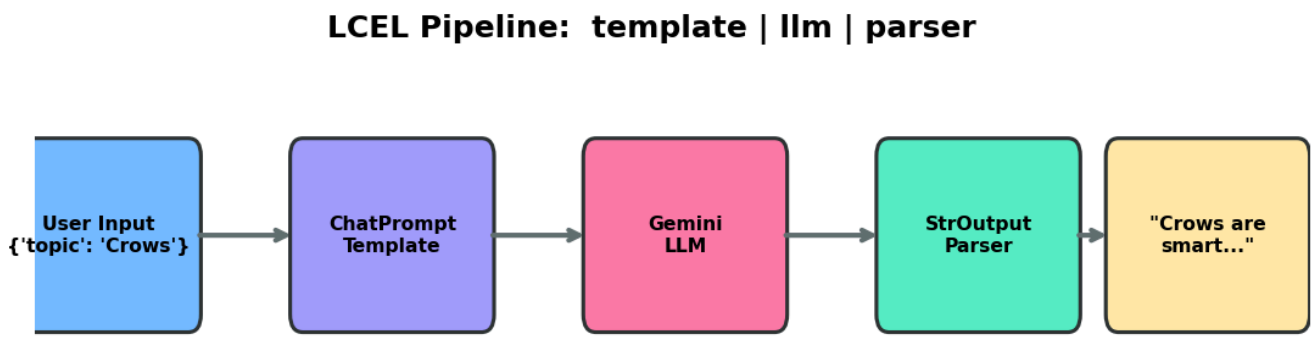
Code:

```
from langchain_core.messages import SystemMessage, HumanMessage

messages = [
    SystemMessage(content="You are a rude teenager. You use slang."),
    HumanMessage(content="What is the capital of France?")
]
response = llm.invoke(messages)
print(response.content)
```

Output:

```
lol its paris duh.. everyone knows that
```

Notebook screenshot: the system message example showed how model personality and style can be controlled.



Notebook visual: LCEL chaining made the input → prompt → model → parser pipeline much easier to understand.

Notebook 1 - LCEL Chain

Code:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

template = ChatPromptTemplate.from_template("Tell me a fun fact about {topic}.")
parser = StrOutputParser()

# LCEL — one line chain!
chain = template | llm | parser
print(chain.invoke({"topic": "Octopuses"}))
```

Output:

```
Octopuses have three hearts and blue blood. Two hearts pump
blood to the gills, and one pumps it to the rest of the body.
```

Notebook screenshot: the LCEL implementation was shorter and cleaner than the manual step-by-step version.

4. Prompt Engineering and In-Context Learning

In the prompt engineering notebook, I saw why structure matters. A vague prompt gives a generic answer because the model fills in missing details with probability. A structured prompt adds context, constraints, and formatting instructions, so the answer becomes more controlled. I also saw the difference between zero-shot and few-shot prompting. Examples help the model copy not just the task pattern but also the tone.

CO-STAR effect

- Better clarity
- Less ambiguity
- More consistent formatting

Few-shot effect

- Better pattern imitation
- Better tone matching
- Higher quality output when examples are good

Notebook 2 - CO-STAR Structured Prompt

Code:

```
structured_prompt = """
# Context
You are an HR Manager at a quirky startup called 'RocketBoots'.

# Objective
Write a rejection email to a candidate named Bob.

# Constraints
1. Be extremely brief (under 50 words).
2. Do NOT say 'we found someone better'. Say 'the role changed'.
3. Sign off with 'Keep flying'.
"""
print(llm.invoke(structured_prompt).content)
```

Output:

```
Hi Bob,

Thanks for taking the time to chat with us. Unfortunately,
the role has shifted direction since we last spoke. We truly
appreciate your interest in RocketBoots.

Keep flying,
The RocketBoots Team
```

Notebook screenshot: structured prompting gave more controlled output than a lazy prompt.

Notebook 2 - Few-Shot Prompting

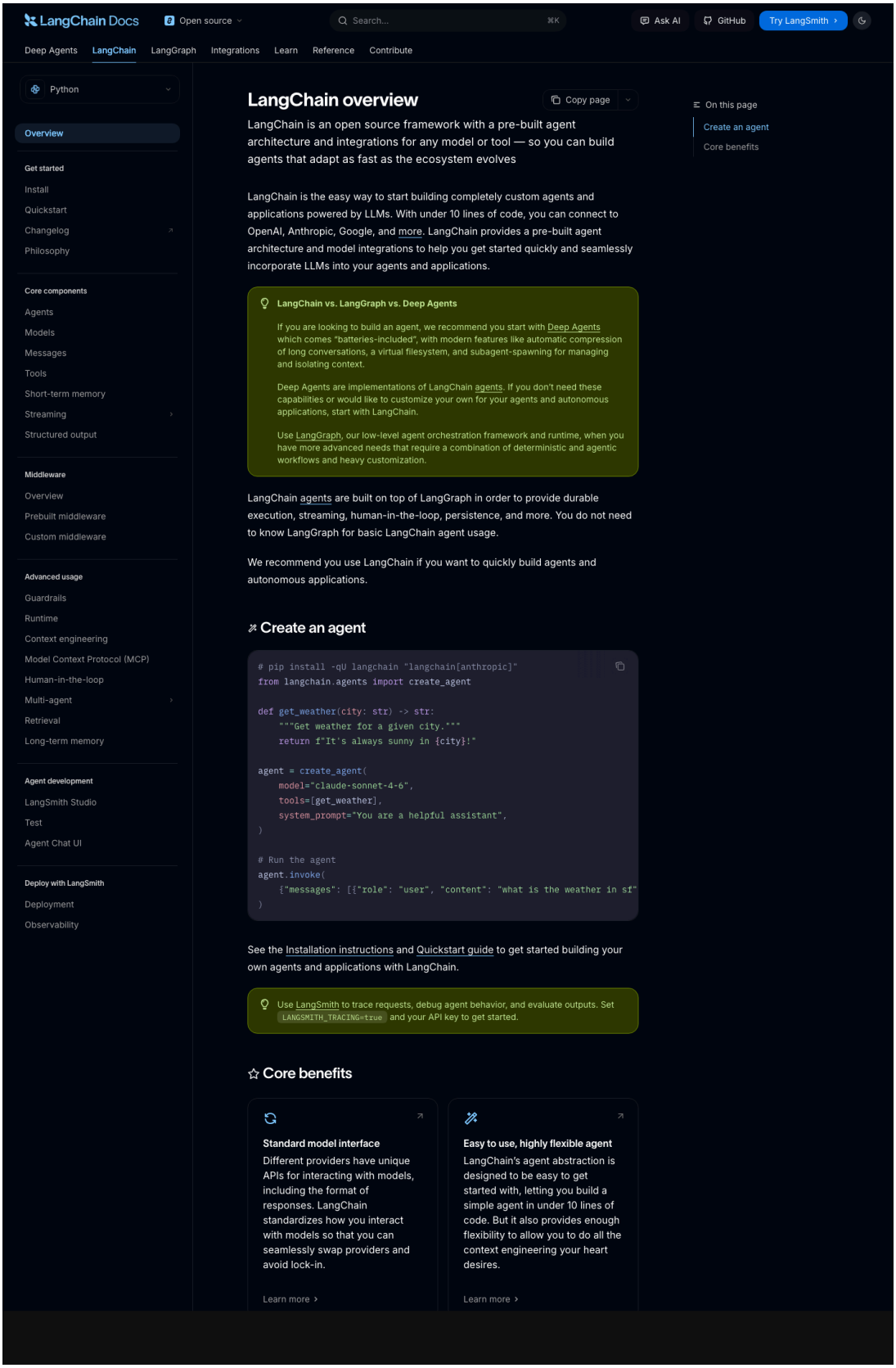
Code:

```
prompt_few = """
Combine words into a funny new word. Give a sarcastic definition.

Input: Breakfast + Lunch
Output: Brunch (An excuse to drink alcohol before noon)

Input: Chill + Relax
Output: Chillax (What annoying people say when you have panic attacks)

Input: Angry + Hungry
Output:
"""
print(llm.invoke(prompt_few).content)
```

Output:

```
Hangry (The emotional state that justifies yelling at loved
ones because you missed a snack)
```

Notebook screenshot: few-shot prompting showed how examples steer both style and structure.

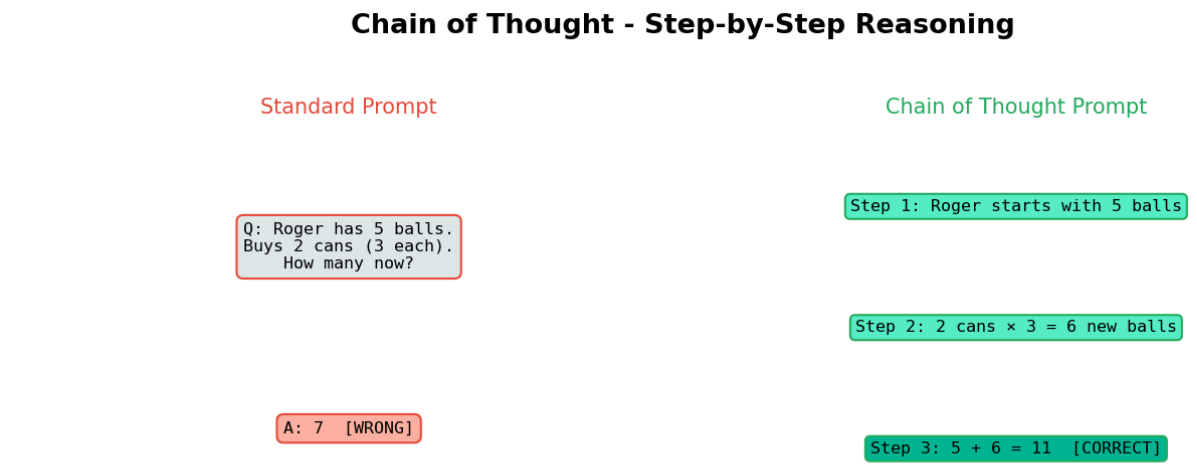


Web screenshot: official LangChain documentation used as an external reference visual for the framework concepts covered in Unit 2.

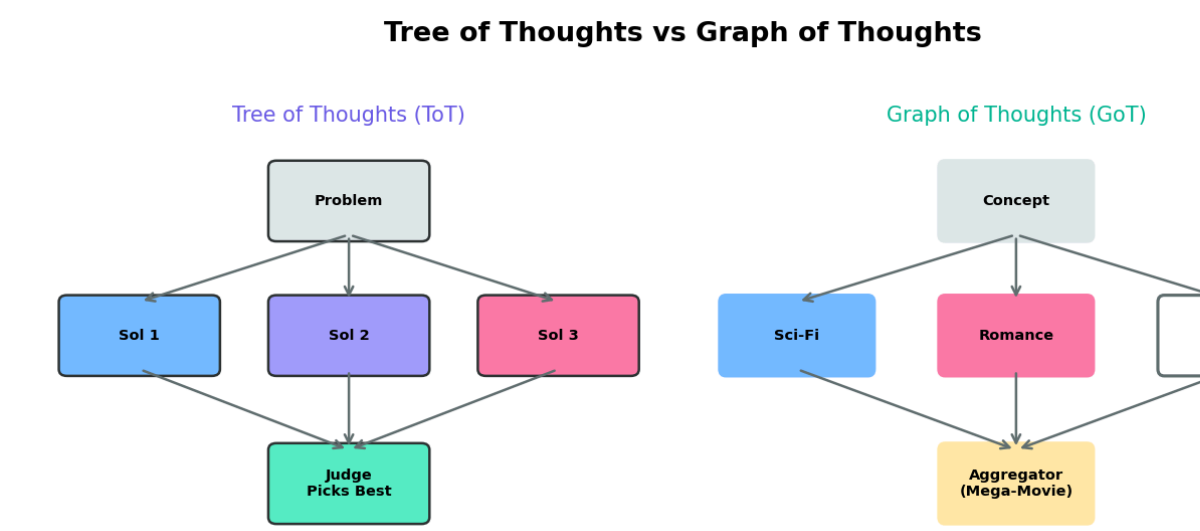
5. Advanced Prompting: CoT, ToT, and GoT

This part helped me separate simple prompting from reasoning-oriented prompting. Chain of Thought is useful when a problem has multiple steps and the model benefits from showing intermediate reasoning. Tree of Thoughts expands this by generating multiple options before selecting one. Graph of Thoughts goes further by combining outputs from multiple branches.

- CoT is best when the problem is linear and step-based.
- ToT is useful when there are multiple possible strategies and one must be judged.
- GoT is useful when several perspectives need to be merged into a stronger final answer.



Notebook visual: CoT clearly improved the answer quality on a multi-step reasoning problem.



Notebook visual: this made the difference between branching and aggregation much easier to grasp.

Notebook 3 - Chain of Thought vs Standard

Code:

```
question = "Roger has 5 tennis balls. He buys 2 more cans. Each can has 3 balls. How many?"

# Standard — just asks directly
prompt_standard = f"Answer this question: {question}"
print("STANDARD:", llm.invoke(prompt_standard).content)

# CoT — forces step-by-step
prompt_cot = f"Answer this question. Let's think step by step. {question}"
print("CoT:", llm.invoke(prompt_cot).content)
```

Output:

```
STANDARD: Roger has 7 tennis balls.  (WRONG!)
CoT:
  Step 1: Roger starts with 5 tennis balls.
  Step 2: He buys 2 cans × 3 = 6 new balls.
  Step 3: 5 + 6 = 11 tennis balls.  (CORRECT!)
```

Notebook screenshot: the “let’s think step by step” prompt is a simple but powerful reasoning trigger.

6. RAG, Embeddings, and Vector Search

I learned that embeddings convert text into vectors so semantic similarity can be measured mathematically. This is the foundation of retrieval. RAG then uses that retrieval result as context for the model, which reduces hallucination and helps answer with external data instead of relying only on training memory.

What stood out most to me was that RAG is basically an open-book test. The retriever finds the relevant information, and the LLM uses that information to answer. Without retrieval, the model might guess. With retrieval, it becomes grounded in the provided documents.

Notebook 4 - Embeddings & Cosine Similarity

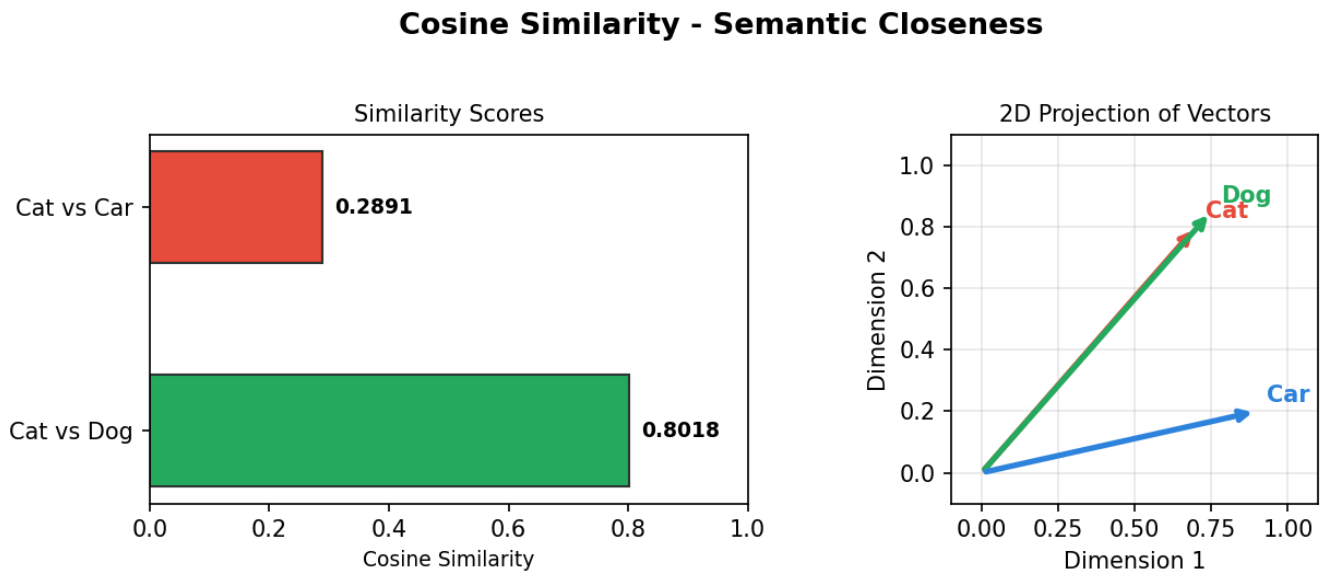
Code:

```
from langchain_huggingface import HuggingFaceEmbeddings
import numpy as np

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

vec_cat = embeddings.embed_query("Cat")
vec_dog = embeddings.embed_query("Dog")
vec_car = embeddings.embed_query("Car")

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print(f"Cat vs Dog: {cosine_similarity(vec_cat, vec_dog):.4f}")
print(f"Cat vs Car: {cosine_similarity(vec_cat, vec_car):.4f}")
```

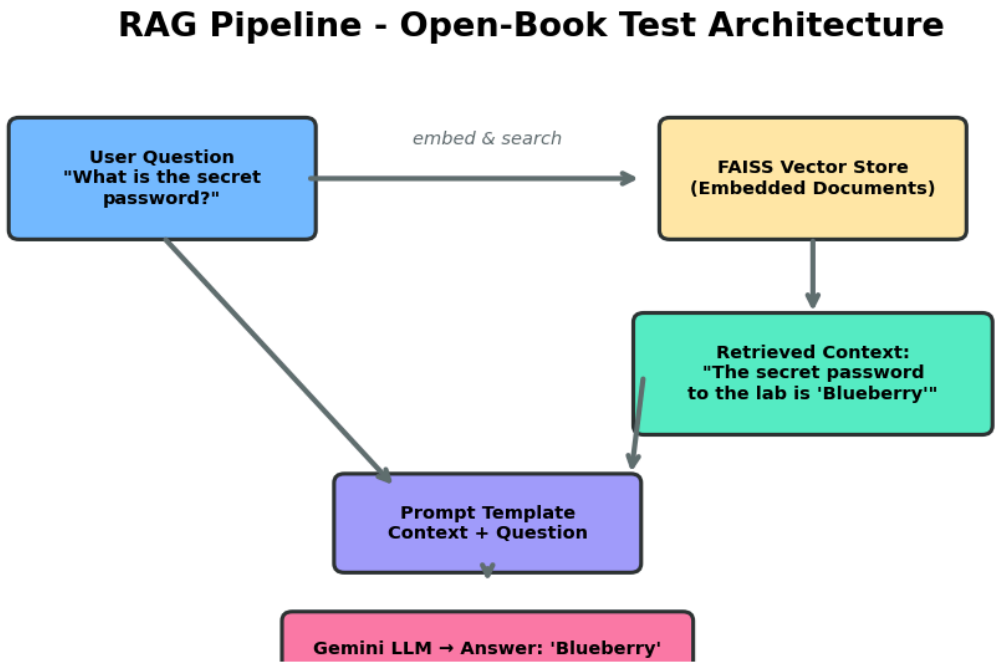
Output:

```
Cat vs Dog: 0.8018
Cat vs Car: 0.2891
```

Notebook screenshot: embedding generation was the first point where text became vector data.



Notebook visual: cosine similarity showed why “cat” and “dog” end up closer than “cat” and “car”.



Notebook visual: the retrieval → augment → generate pipeline captured the logic of naive RAG clearly.

Notebook 4 - RAG Pipeline in Action

Code:

```
from langchain_community.vectorstores import FAISS
from langchain_core.runnables import RunnablePassthrough

docs = [
    Document(page_content="Piyush's favorite food is Pizza with extra cheese."),
    Document(page_content="The secret password to the lab is 'Blueberry'."),
    Document(page_content="LangChain is a framework for LLM apps."),
]
vectorstore = FAISS.from_documents(docs, embeddings)
retriever = vectorstore.as_retriever()

chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)
print(chain.invoke("What is the secret password?"))
```

Output:

```
The secret password is 'Blueberry'.
```

Notebook screenshot: the chain construction showed how the retriever and question are combined into one prompt flow.

7. Indexing and Mixture of Experts

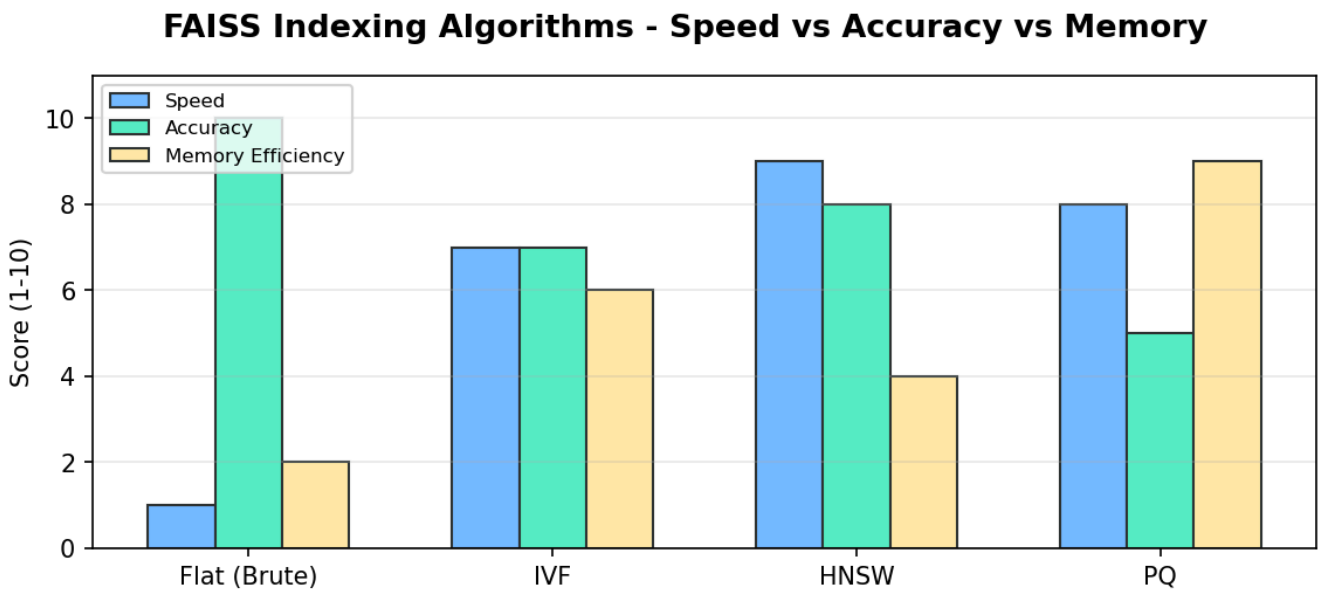
The indexing section clarified why brute-force similarity search is too expensive at scale. Flat, IVF, HNSW, and PQ each optimize a different tradeoff among speed, accuracy, and memory. The final MoE assignment then showed a different idea: not retrieving documents, but routing questions to the right expert behavior.

Indexing insight

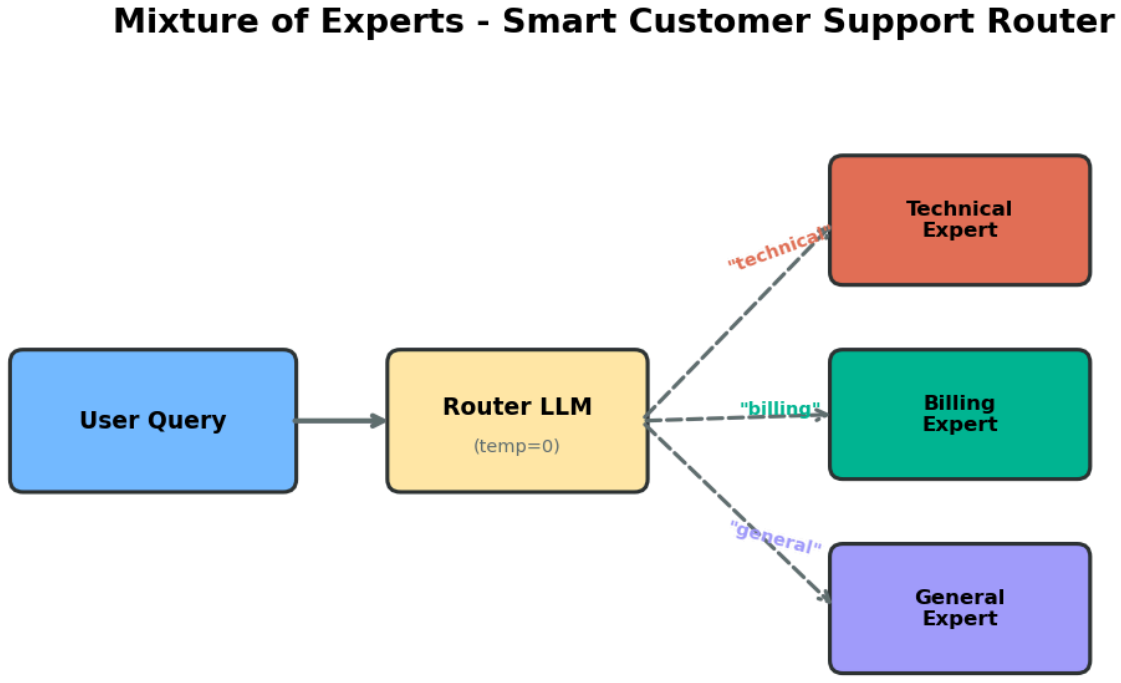
FAISS algorithms are not all trying to do the exact same thing. Some focus on accuracy, some on speed, and some on compression. That tradeoff matters in real systems.

MoE insight

A single base model can behave like multiple experts if the router is reliable and the system prompts are well designed.



Notebook visual: the comparison among Flat, IVF, HNSW, and PQ made the scale tradeoffs easier to remember.



Assignment visual: the MoE router directs technical, billing, general, or tool-style requests to the most suitable path.

Assignment - MoE Router Function

Code:

```
def route_prompt(user_input: str) -> str:
    response = client.chat.completions.create(
        model=BASE_MODEL,
        messages=[
            {"role": "system", "content": ROUTER_SYSTEM_PROMPT},
            {"role": "user", "content": user_input},
        ],
        temperature=0,       # deterministic routing
        max_tokens=10,
    )
    category = response.choices[0].message.content.strip().lower()
    if category not in VALID_CATEGORIES:
        category = "general"  # fallback
    return category
```

Output:

```
Router Test Results:
  "IndexError on line 5"  -> [TECHNICAL]
  "Charged twice"         -> [BILLING]
  "Hey, what's up?"       -> [GENERAL]
```

Assignment screenshot: the router logic and orchestrator showed how classification and response generation work together.

8. Final Reflection

Unit 2 felt like the transition from “using an LLM” to “engineering an LLM application.” At the start, the focus was on model setup and prompt basics. Then it moved into structured prompting, chaining, reasoning patterns, retrieval, indexing, and routing. The important shift for me was understanding that good GenAI systems are pipelines, not single prompts.

My strongest takeaways are:

- prompt quality directly affects output quality,
- reasoning structure matters when tasks are complex,
- retrieval helps with freshness and grounding,
- routing helps specialize behavior without changing the base model.

These notes are based on the Unit 2 notebooks, experiments, and assignment work completed during the hands-on sessions.